Assignment 3: Implementat Transaction Management, Concurrency Control, and ACID Validation on Application and B+ Tree Database

1. Project Objective

The objective of this assignment focuses on making your system reliable and correct under both normal and heavy usage.
In **Module A,** Ensure correct behaviour of transactions and Crash Recovery natively within your custom B+ Tree and Database Manager.
In **Module B,** you will Ensure the application works safely when many users use it together

Core Technical Pipeline:

Module A: Transaction Behaviour & Crash Recovery (The "Engine")

- Ensure correct execution of transactions (complete or rollback)
- Implement logging and recovery for failure handling
- Maintain consistency between database records and B+ Tree index

Module B: Concurrent Workload & Stress Testing

- Simulate concurrent operations using multi-threaded scripts (tools optional)
- Test system behaviour under failures and rollback scenarios
- Observe performance and correctness under high load

Deadline

6:00 PM, 5 April 2026

Instructor: Dr. Yogesh K. Meena
March 23, 2026 • Semester II (2025 - 2026) • CS 432 – Databases(Course Project/Assignment 3)

1. Assignment Overview

This assignment consists of two modules (Module A and Module B).

- **Module A:** Advanced Transaction Engine & Crash Recovery (The "Engine").
- **Module B:** High-Concurrency API Load Testing & Failure Simulation (The "Stress Test").

2. Module A: ACID Validation (Correctness of Operations)

Objective

In this module, you will extend the B+ Tree-based mini-database system developed in Assignment 2 to support transaction management, failure recovery, and ACID guarantees. The focus is not on rebuilding the database, but on making your existing system reliable, consistent, and robust under failures and concurrent execution.

Existing System (From Assignment 2)

From your assigment 2 you already have developed:

- A custom database manager
- Table abstraction
- B+ Tree-based storage for records

Important:

- The B+ Tree already acts as the primary storage structure for your data.
- All records are stored and accessed through the B+ Tree.

Database Requirement

Your system must contain at least three relations (tables) from Assignment 2.
Example (illustrative only):

- Users(user_id, name, balance, city)
- Orders(order_id, user_id, amount, time)
- Products(product_id, name, stock, price)

Requirement:

- Each relation must continue to be stored using a separate B+ Tree.
- The primary key must be used as the B+ Tree key.
- The value must represent the complete record.

Role of B+ Tree (Clarification)

In this assignment, the B+ Tree is:

- The storage engine for each relation
- The indexing structure
- The only access path for all operations

Not allowed:

- Maintaining a separate copy of data outside the B+ Tree
- Using the B+ Tree only as an auxiliary index

Consistency Interpretation

The requirement:

> "Data in the database and B+ Tree must match"

should be interpreted as:

- The B+ Tree is the database representation.
- All operations must directly modify the B+ Tree.
- At no point should partial or inconsistent records exist within any B+ Tree.

Transaction Requirements

You must extend your system to support transactions across multiple relations (B+ Trees). Each transaction must support:

- BEGIN
- COMMIT
- ROLLBACK

ACID Requirements

Atomicity

- A transaction involving multiple tables must either:
  - complete fully, or
  - be completely rolled back
- No partial updates should remain after failure

Consistency

- After each transaction, all relations must remain valid
- Constraints such as valid references and non-negative values must hold

Isolation

- Concurrent transactions should not corrupt shared data
- Basic locking or serialized execution is sufficient

Durability

- Once committed, data must persist across system restarts

Multi-Relation Transaction Requirement

Important:

- ACID validation must be demonstrated using transactions that operate on at least three relations.

Example scenario:

- Update a user's balance
- Update product stock
- Insert a new order

All three operations must be part of a single transaction.

Failure Handling and Recovery

- Simulate failures during transaction execution
- Ensure:
  - Partial updates are rolled back
  - Committed data is preserved
- After restart:
  - Undo incomplete transactions
  - Retain committed transactions

What to Test

- **Atomicity:** Crash during a multi-table transaction and verify rollback
- **Consistency:** Ensure all relations remain valid after operations
- **Isolation:** Execute concurrent transactions on the same data and verify that intermediate states are not visible and no data corruption occurs
- **Durability:** Restart system and verify committed data persists

3. Module B: Multi-User Behaviour and Stress Testing

This module focuses on how your system behaves when many users use it together.

Concurrent Usage

- Simulate multiple users performing operations at the same time
- Try accessing and modifying the same data
- Ensure users do not interfere with each other

Race Condition Testing

- Identify a critical operation (e.g., booking, update)
- Simulate many users trying the same operation
- Ensure no incorrect results occur

Failure Simulation

- Introduce failures during execution
- Ensure system rolls back correctly
- Verify no partial data is stored

Stress Testing

- Run a large number of requests (hundreds or thousands)
- Observe system behaviour under load
- Check correctness and response time

You may use tools like Locust, Apache JMeter, or your own scripts.

What to Verify

- **Atomicity:** Operations fully complete or fully rollback
- **Consistency:** Data remains correct
- **Isolation:** Users do not affect each other
- **Durability:** Data persists after failure

Submission

- **Report:** `group_name_report.pdf`
- **Short Video Demonstration**

Report Requirements

First page must include:

- GitHub repository link
- Video link

The report should explain:

- How correctness of operations is ensured
- How failures are handled
- How multi-user conflicts are handled
- What experiments were performed
- Observations and limitations

Video Requirements

- Show your system running
- Demonstrate concurrent usage
- Show failure and recovery
- Explain behaviour clearly

Evaluation Criteria

- Correctness of transaction behaviour
- Proper handling of failures
- Multi-user safety and isolation
- Consistency between the database and the B+ Tree
- System robustness under load
- Clarity of explanation

Conclusion

The goal is to build a system that works correctly, handles failures safely, and supports multiple users without breaking. The focus is on making your system robust and reliable.